arXiv:2104.03221v1 [cs.DS] 7 Apr 2021

面向缓存优化近邻搜索的图重排序

本杰明·科尔曼[1 *] 圣地亚哥·塞加拉[1] 亚历克斯·斯莫拉[2] 安舒马利·什里瓦斯塔瓦[1 3]

摘要

图搜索已成为近邻搜索领域中最成功的算法趋势之一。其中若干广受欢迎且经实证验证有效的算法，其核心机制均在于沿已剪枝的近邻图进行简单遍历。此类算法在嵌入搜索等应用场景的工业级性能基准测试中，始终稳居顶尖梯队。然而，图遍历类应用常面临内存访问模式欠佳的问题，近邻搜索亦不例外。

我们的测量结果表明，诸如分层可导航小世界图（HNSW）等常用搜索索引可能具有较差的缓存未命中性能。为解决这一问题，我们将图重排序算法应用于近邻图。图重排序是一种内存布局优化技术，它将频繁访问的节点在内存中集中放置。我们进行了详尽的实验，将多种重排序算法应用于基于 HNSW 索引的主流图基近邻方法。实验发现，重排序可将查询时间最多缩短40%，并且与构建索引所需时间相比，重排序所需时间可忽略不计。

1. 引言

近邻搜索是机器学习系统中众多应用场景的基础构建模块。通俗地说，该任务可理解如下：给定数据集$D=\{x_1, x_2, ...x_N\}$，我们需要构建一种*数据结构*，使得该数据结构支持对任意查询点q进行查询，从而获取与查询点距离最近的k个数据点$x_i \in D$。这种数据结构被称为*近邻索引*。近邻搜索指标具有极其重要的实际意义，因为它们构成了推荐系统、自然语言处理（Mikolov 等人，2013）、基因组学（Ondov 等人，2016）、计算机视觉（Lowe，1999）以及机器学习领域其他应用（Shakhnarovich等人，2008）中生产模型的基石。

应用场景。近邻搜索始终是经典学习算法（如基于邻域的分类与回归）的核心组成部分。然而，由于其在嵌入模型预测任务中的关键作用，该问题近来已成为密集研究关注的焦点。在神经嵌入模型中，对象被转换为*嵌入向量*，其维度为^d，其中d的取值范围为100至1000，而N通常超过1亿。预测任务通常包括寻找k个最近邻嵌入向量。例如，亚马逊的深度语义搜索引擎通过将产品目录与搜索查询嵌入到同一高维空间来实现功能——该引擎会推荐那些嵌入向量与搜索查询最接近的产品（Nigam等，2019）。

由于每次查询都会进行搜索，机器学习系统的延迟与召回率关键取决于在高召回率场景下快速执行近邻搜索的能力。类似问题也存在于自然语言处理（Kusner等，2015）、信息检索（Huang等，2013）、计算机视觉（Frome 等，2013）及其他应用领域。除机器学习外，近邻索引还被用于音频录制匹配、抄袭检测、医疗领域的心电图信号分类，以及结合区块链实现版权归属判定（Jafari等，2021）。因此，近邻搜索已成为众多应用中的核心瓶颈。

Latency Challenges .近邻问题的解决方案极为多样，从硬件加速的暴力搜索（Johnson 等人，2019）到空间划分树（Beygelzimer 等人，2006）不等。图算法已成为高维近邻搜索中一类显著有效的方法。例如，在成熟的 ANN 基准测试排行榜中，前五名搜索库中有四个采用了图索引（Aumu ... ller等人，2020）。由于这些库已被集成到众多大规模生产系统中（Johnson 等人，2017；Malkov & Yashunin，2018），它们已在 C++

[*] 部分研究工作是在作者任职于亚马逊期间完成的。[1]德克萨斯州休斯顿市莱斯大学电气与计算机工程系[2]亚马逊云服务[3]德克萨斯州休斯顿市莱斯大学计算机科学系。通讯作者：Benjamin Coleman <ben.coleman@rice.edu>、Anshumali Shrivastava <anshumali@rice.edu>、Alex Smola <alex@smola.org>。

版权所有 © 2021 作者。

中经过大量工程成本的手工优化。快速近邻搜索系统已成为一个高度活跃的研究领域焦点，其优化措施具有明确的实际效益。

在本文中，我们研究了利用*图重排序*来改进近似近邻搜索的可能性。图重排序是一种缓存优化技术，其原理是将相邻节点置于连续的（或近乎连续的）内存位置中。当图处理算法将某个节点加载到内存时，现代CPU架构会自动从邻近的内存位置加载其他节点。通过重排序这些节点，我们可以确保算法中可能访问到的节点已被预先加载到CPU缓存中。重排序过程看似简单，但对于PageRank、图分解及图直径计算等任务而言，RAM访问量可占运行时间的70%以上（Wei等，2016）。由于缓存命中速度比内存访问快一个数量级，重排序可将图相关应用的运行速度提升50%以上（Balaji & Lucia，2018）。我们的研究证明，通过图重排序，近邻搜索可显著加速——这一针对重要机器学习任务的优化方法此前尚未被探索。

1.1. 我们的贡献

我们将六种最新的图排序算法整合到分层可导航小世界图（HNSW）中，这是一种领先的图索引技术，在常见基准测试中展现出最先进的性能。我们在 SIFT 100M、GIST 和 DEEP100M 数据集上进行了全面比较，评估了查询时间和缓存未命中率。实验结果表明，图重排序是一种可行且有效的方法，可提升涉及近邻搜索的实际机器学习部署性能。我们得出以下新发现：

- 图重排序可显著缩短近邻搜索的查询时间，在大型嵌入数据集上可实现 10%-40% 的速度提升。
- 基于目标函数的技术通过优化节点顺序以最大化全局缓存一致性评分，因而具有最佳性能。而近期提出的仅利用局部节点特征的*轻量级*图算法，则无法提升近邻搜索性能。
- 尽管基于目标的方法属于成本最高的重排序算法之一，但其重排序时间通常比构建搜索索引所需时间少一个数量级。

重新排序是否能提升近邻搜索性能这一问题并非显而易见，因为根据输入图的特性及应用程序的节点访问模式，重新排序实际上可能*降低*图处理效率（Balaji & Lucia，2018）。需要指出的是，近邻图与通常能从重新排序中获益的图具有截然不同的结构特性。例如，重新排序算法在具有幂律度分布的图上常能获得显著加速效果（Faldu 等，2019），而近邻图并不具备这一特性。此外，重新排序搜索索引的过程是否代价过高亦尚不明确。对于重新排序问题的常见建模方式而言，寻找最优节点标记函数是一个 NP 难问题（Wei 等，2016）。尽管存在近似算法与启发式方法，但其时间复杂度与节点度数之和呈二次方增长。对于社交网络这类图结构而言，这一问题并不突出——由于其遵循幂律分布，平均度数较低；但在对连接度更高的近邻图进行重排序时，则可能引发严重问题。在工业级嵌入搜索场景中，图索引常需根据模型变化进行重构。因此，至关重要的是：我们的性能提升不会显著增加索引构建耗时。

2. 背景与相关工作

近邻图。基于图的算法，如 HNSW 、剪枝近似近邻图（PANNG）（Iwasaki，2016）以及优化近邻图（ONNG）（Iwasaki & Miyazaki，2018），在Web规模企业的排行榜和生产系统中占据重要地位。除了最近的理论进展外（Prokhorenkova & Shekhovtsov，2020），图索引研究的主要焦点一直是开发启发式方法（如多样化、剪枝和分层结构），以改善图的特性、提高搜索精度并缩短搜索时间。在第3节中，我们证明大多数搜索算法的计算工作量基本相同，因为这些启发式方法生成的图结构高度相似。

减少搜索时间的另一种方法是开发可用于构建高速系统的实现技术。鉴于其在工业领域的重要性，搜索索引已得到了大力优化。然而，尽管已知图算法的缓存性能较差，但针对改进基于图的近邻搜索的缓存局部性特性的研究却出奇地少。Boytsov & Naidan（2013）将内存碎片化列为 HNSW 设计中的一个重要考量因素，而 HNSW 软件手册（Naidan 等，2015）则介绍了一种“扁平化索引”，以优化近邻搜索的内存访问模式。这种扁平化布局将节点与数据共同存储于内存的连续块中，无需使用指针。这种高效布局可提升缓存

性能，但对图结构并无依赖性。图重排序则是一种兼容技术，它利用图的相关信息进一步优化布局。

基于空间填充曲线的排序。与图重新排序最相关的思路可能是将空间填充曲线用于搜索问题（Chan，2002）。空间填充曲线是一条穿过数据集中每个点的直线。通过最小化这条直线的长度，我们可以获得一种点排序方式，使得邻近的点位于连续的位置。这一思路主要被用于通过线性扫描（已排序的）数据来实现缓存高效的近邻搜索，但也已应用于图结构。Connor & Kumar（2010）采用莫顿排序（一种特定类型的空间填充曲线）来提升三维点云图构建的缓存性能。然而，这项研究并不适用于机器学习应用，例如嵌入搜索，原因有二：首先，空间填充曲线受限于维度灾难，对于嵌入搜索中常见的高维向量排序而言并非有效手段。其次，现有研究采用空间划分曲线与重排序技术来加速图的*构建*过程，而我们的研究重点在于重排序是否可缩短*查询时间*。据我们所知，本文是迄今唯一针对快速近邻查询而开展图重排序研究的工作。

3. 图搜索索引的核心组件

在本节中，我们将介绍基于图搜索索引的核心要素。正如后续内容所示，流行的基于图的索引（HNSW、PANNG 和 ONNG）都具有相同的核心组件。因此，尽管我们在实验中专注于 HNSW，但我们预期这些发现可推广至实践中使用的大多数基于图的索引。从实现角度来看，图索引的核心组件包括*多样化/剪枝*、*搜索初始化*以及*束搜索*。尽管主要搜索算法在非平凡方面存在差异，但这些差异并不会影响应用程序的节点访问模式，或与图重新排序相关的图属性。

在近邻图索引中，每个节点对应数据集中的一个元素。若两个节点对应的元素之间距离较小，则这两个节点相连。据我们所知，所有现有的近邻图理论模型均采用以下两种构建方式之一（Laarhoven，2017；Sebastian &Kimia，2002），其中我们假设每对节点间都存在一个度量$d:V \rightarrow V$。

定义 1若对于任意两个节点v_1与v_2，均存在一条边连接它们，且满足$d(v_1, v_2) < \alpha$，则称该图为α-*近邻（*α-*NN）*图。

定义 2若每个节点v均与其k个最近节点相连，则该图称为k-*最近邻 (k-NN)*图。

这两种图模型在度分布与连通性方面存在差异。在k-NN图中，每个节点的出度均等于k；而在α-NN图中，节点可连接任意数量的邻居。此外，k-NN图是有向图，而α-NN图是无向图——这是因为距离函数d具有对称性。需注意的是，两种模型均可能生成不连通图：数据集中分布稀疏区域的节点会形成独立的连通分量。

3.1. 构建近邻图

在实践中，我们无法高效构建k-NN或α-NN图，因为为找到数据集中每个点的最近邻，需要执行$O(N^2)$次计算。几乎所有现代算法都通过“自启动”策略来解决这一鸡生蛋问题——即迭代地将节点添加到部分构建完成的图中。若要将节点q加入图，我们需查询当前版本的图以获取q的（近似）邻域，随后将边的集合更新以包含q。最终得到近似k-NN或α-NN图。另一种替代基于搜索的构建方法是：从随机初始化的图出发，应用名为*NN-Descent*的流程——对图中每个节点进行查询与更新（Dong等人，2011）。由于该技术仅需少量迭代即可收敛至真实图的近似解，故也可用于*优化*通过其他方法获得的图。

过去，诸如 SWG 等高性能算法直接使用图结构（Malkov 等人，2014）。然而，当前最快的索引算法已不再使用原始的k-NN 或α-NN 图。相反，最终搜索索引是通过移除不必要的边（或称*剪枝*）以及添加边（或称*多样化*），以提升图的可导航性而构建的。每种图索引实现这种可导航性提升所采用的方法略有不同，但它们均遵循源自Arya & Mount（1993）的相似原理。下文我们将介绍来自 ANN-benchmarks 排行榜前列的若干算法。

HNSW可以说是最受欢迎的图索引，已被广泛应用于工业领域（Malkov & Yashunin，2018）。HNSW 索引由一个分层图构成，其中每一层包含来自数据集的 progressively 更多数据点。第l层是一个对第l_1 层所有节点进行剪枝后的k-NN 图，并额外添加了$O(el)$ 个未出现在前一层中的新节点。由于各层节点数呈指数级增长，上层包含的节点相对较少，而最底层则是完整数据的k-NN 图。HNSW 的核心理念是：通过考察上层结构，可快速确定一个适用于底层进一步探索的优质邻域。HNSW 直接采用了 Arya & Mount

（1993）提出的剪枝启发式方法，具体如下：假设节点B是节点A的k个最近邻节点之一。如果节点B与A的其他邻居中某个节点的距离，比其与A的距离更近，则剪除A与B之间的连接边。

PANNG 与 ONNG 。PANNG 会对k-NN 图进行剪枝操作：当两个节点间存在替代路径时移除相应边（Iwasaki，2016）。 PANNG 算法首先向k-NN 图添加边，使该图成为完全连通且无向的图；随后，该算法依据以下剪枝启发式规则移除边：给定两个节点A与B，若B不在A的前k_r个邻居之列，且A与B之间存在长度$\leq p$的不同路径，则移除该边。其中k_r与p为超参数。当p= 2 且k_r= 0 时， PANNG 启发式算法剪枝的边集与 HNSW 启发式算法相同，唯在实践中罕见的边缘情形下例外[1]。 ONNG 在 PANNG 基础上引入额外多样化步骤，确保每个节点的入度至少为k_I，随后采用p= 2 的相同启发式规则（Iwasaki & Miyazaki，2018）。换言之， ONNG 首先确保所有节点均可达，然后从图中所有三角形中移除最长边。

Other Algorithms .其他若干研究成果，例如 FANNG（Harwood & Drummond，2016）以及基于 NN-Descent 的方法，都曾一度成为最快搜索算法的有力竞争者。这些方法所采用的剪枝技术与上述所述技术相似。关于最新综述及详尽讨论，读者可参阅Shimomura 等人（2020）的文献。

图指标之间的相似性。大多数图算法都从在节点间添加“长距离”边开始，然后在找到更理想的替代路径时剪掉这些边。在常见的超参数选择下，这些剪枝启发式方法的表现都相当相似。由此生成的图在实际搜索任务中展现出近似相同的性能（Aumu ... ller等人，2020），这并不令人意外。如前所述，我们预期这些相似性将推动我们的研究成果（此处以 HNSW 为例）基于相同原理推广至其他常用图指标。

3.2. 近邻图搜索

基于图的近邻搜索包含沿图边进行的遍历过程。在遍历过程中遇到的每个节点x处，我们记录查询项q与当前节点之间的距离$d(q, x)$。我们持续遍历该图，直至抵达一个与查询项距离比迄今为止见过的任何候选节点或候选节点的邻近节点都更近的节点。

[1]当从A到B的路径上的中间节点C不是B的邻居时，这种情况会发生。若B与C是A的邻近节点，则这种情况较为罕见，因为$d(B, C) \leq d(A, B) + d(A, C)$。

束搜索。目前广泛使用的两种搜索算法是：贪婪搜索与束搜索[2]。在贪婪搜索中，我们仅从当前节点的邻居中选取与查询节点距离最近的节点；一旦所有出边均无法指向比当前节点更近的节点，则终止搜索过程。束搜索在概念上与贪婪搜索类似，但我们会维护一个包含M个候选节点的动态列表以供探索。束搜索允许我们从迄今发现的任意M个最佳节点出发，探索其出向路径。若第i个候选节点的出边均无法指向更近的节点，则转而考察候选节点（i+1）的邻居。当M个候选节点均无指向更近邻居的边时，我们将候选列表中的最优节点作为搜索结果返回。需注意：当M=1时，束搜索与贪婪搜索等价。

束搜索初始化。近年来图k-NN 领域的许多创新均可归结为*初始化方法*，这些方法能为束搜索提供良好的初始位置。例如， HNSW 在图的层次结构部分进行贪心搜索，以在最低层中找到一个种子节点，该节点随后用于初始化束搜索； PANNG 与 ONNG 则通过遍历空间划分树来提取种子节点。其他可选方案包括基于局部敏感哈希的初始化、节点中心性度量，甚至随机选择。近期实验表明，初始化方式对性能并无显著影响（Lin & Zhao，2019a；b）。Prokhorenkova & Shekhovtsov（2020）通过证明：在α-NN 图上进行束搜索，只要采用可通过重复随机采样获得的“足够接近”的初始化，即可解决近似近邻问题，从而为这一观点提供了理论支持。Laarhoven（2017）则针对贪心搜索证明了相关结论，其基础亦依赖于类似的初始化方式。

初始化过程的成本也相当低。为了说明这一点，我们在 SIFT1M 数据集上构建了一个 HNSW 图，每个节点包含k= 8 条边。我们使用M= 500 的束搜索方法，发现搜索过程中的分层部分仅占查询时间的 2.9%（± 1.9%）。随后，我们将分层结构替换为一种新流程：随机选择 50 个节点，并以最近的选项作为初始化点，以此复现Lin & Zhao（2019a）的实验。在 10,000 次查询中，我们在召回率或查询时间方面均未发现统计学上的显著差异。实际应用中，在典型查询条件下，初始化过程虽然是搜索流程中的重要环节，但其计算开销微乎其微。初始化方法的不同不会影响实验结果的有效性。

[2]我们采用Prokhorenkova &Shekhovtsov（2020）提出的束搜索定义。图搜索相关文献则采用略有不同的定义，即探索每个候选节点（而非仅探索最佳节点）。

4. 图重排序

在本节中，我们介绍了应用于近邻图的图重排序方法，其目的在于加速搜索过程。从形式上讲，图重排序可视为构建一个标记或索引函数$P:V \rightarrow \{1, \ldots, N\}$，该函数根据预设规则，或为最大化某目标函数，将图中的每个节点$v \in V$映射至1至N之间的唯一整数索引（或标签）。尽管可能存在多种表述方式，但通常我们要求P将连通节点映射至相似（邻近）的标签。随后，函数P被用作图的内存布局，其中节点v被分配至内存位置$P(v)$。

4.1. 技术方法

我们考虑三类主要的重排序算法。第一类算法旨在优化P以最大化目标函数；该优化问题通常属于NP难问题，因此我们通常只能接受P的近似解。第二类算法根据节点的度数或其他局部图特征为节点分配标签。最后，图划分算法可通过为每个分区分配连续内存区间，被重新用于实现重排序功能。下文我们将逐一回顾每类算法中的若干最新研究成果，这些算法将在第5节中接受评估，以检验其在加速近邻搜索方面的效能。

Gorder。Gorder（图排序）是一种图重排序算法，旨在最大化具有连续标签的节点邻域之间的重叠度（Wei等人，2016）。实际上，Gorder 通过最大化由连续节点组成的大小为w的块中共享边的数量，来找到P。这是衡量缓存效率的一个良好指标，因为包含大量重叠节点的块更有可能避免缓存未命中，毕竟每个节点的邻居都存储在不超过w个内存位置范围内。形式上，Gorder 的标记函数P_{GO}可通过以下最大化过程求得。

$$P_{\mathrm{GO}} = \arg\max_{P} \sum_{\substack{u,v \in V \text{s.t.} \\ |P(u)-P(v)|<w}} S_s(u,v) + S_n(u,v), \quad (1)$$

其中$S_s(u, v)$表示u与v是否直接相连，而$S_n(u, v)$则统计它们拥有多少个公共邻居。换言之，Gorder算法旨在最大化在标记函数下彼此间距在w个位置范围内的节点之间的*平均邻域重叠度*。直观而言，若两个节点共享一条直接边，或即使不共享直接边但拥有大量公共邻居，它们仍会被归为一组。最大化公式(1)中的目标函数属于NP难问题，但Wei等人(2016)提出的Gorder算法可提供1/(2w)-近似解。

逆 Cuthill-McKee 算法。矩阵的*带宽*定义为非零元素与主对角线之间的最大距离。逆 Cuthill-McKee（RCM）算法（Cuthill & McKee，1969；George，1971）是一种重排序方法，最初用于最小化稀疏对称矩阵的带宽。若将此方法应用于邻接矩阵，则相应的优化问题在图重排序背景下可直接解释（Auroux 等，2015）。

$$P_{\mathrm{RCM}} = \arg\min_{P} \max_{(u,v)\in E} |P(u) - P(v)|, \quad (2)$$

其中E表示目标图的边集。由式（2）可得，RCM 的目标是最小化连通节点间的*最大标签差异*。与 Gorder 的方法类似，该问题具有较高难度：其精确解属于 NP-完全问题。RCM 算法是一种基于广度优先搜索的启发式方法，用于寻找具有较低（但未必最优）带宽的函数P。

MLOGA 与 MLINA 。公式（2）中的目标促使研究者开发了专注于连接节点标签间差异的聚合度量（而非最大值）的扩展方法。最初源于社交网络压缩领域的最小对数排列（MLOGA）与最小线性排列（MLINA），均是此类方法的典型示例（Chierichetti等，2009）。更具体地说， MLINA 旨在最小化标签差异的总和。

$$P_{\mathrm{MLN}} = \arg\min_{P} \sum_{(u,v)\in E} |P(u) - P(v)|, \quad (3)$$

而 MLOGA 则首先对这些差异进行对数变换。

$$P_{\mathrm{MLG}} = \arg\min_{P} \sum_{(u,v)\in E} \log(|P(u) - P(v)|), \quad (4)$$

其中，E仍是目标图的边集。正如预期，这两个问题均属于 NP-难问题。然而，已开发出专门的启发式算法，用于近似求解 MLOGA（Chierichetti 等人，2009；Safro & Temkin，2011）与 MLINA（Wei 等人，2016）问题。

度数排序。度数排序是一种轻量级重排序算法，其核心思想是：高度数节点很可能共享大量边。事实上，许多实际图中的节点度分布均遵循幂律分布——即少数节点构成一个密集连接的子图。对于具有幂律度分布的无向图，度数排序可在首个连续内存块中构建一组相邻的高度数节点组。为获取P，我们只需按节点度数降序对节点进行排序，并令$P(v)$表示节点v的排序秩。由于度数排序仅需局部节点信息，其运行速度较公式(1)-(4)中的基于优化的方法快数个数量级。

Hub Sorting .枢纽排序（Zhang 等人，2017）与度数排

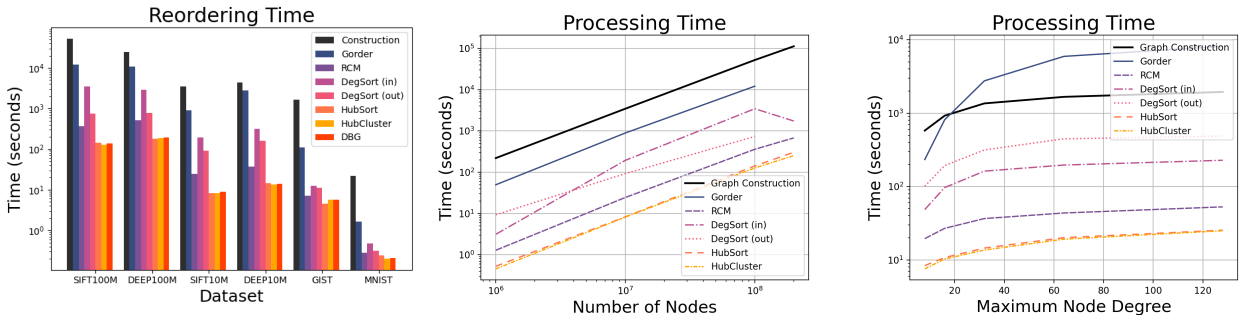


图 1。图重排序算法可适用于大规模数据集及复杂的近邻图。在我们的实验中，重排序算法的运行时间短于图构建过程（左图）。即使对于包含大量节点的大规模图（SIFT，k= 16，中图），以及针对大规模数据集上的密集连接图（$k \leq 120$，右图），重排序操作均可行。

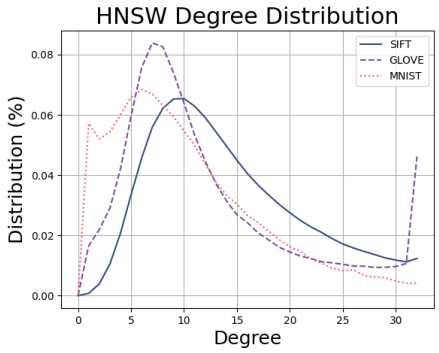


图 2。在实际应用中，剪枝后的k-NN 图远非k正则图。该图展示了三种图索引中 HNSW 基础层的经验出度分布。尽管我们初始设定每个节点拥有k= 32 条出边，但剪枝启发式算法仍从网络中移除了超过一半的边。

序类似，但需注意我们仅对枢纽节点（具有大量连接的节点）进行排序。该算法首先根据度数阈值将节点划分为两组（枢纽节点与非枢纽节点）。该阈值设定为平均度数，从而将度数高于平均值的节点定义为枢纽节点。为计算P，先将枢纽节点按度数排序，非枢纽节点则保持原有顺序。实验表明，度数排序与枢纽排序并非总是有效：若图结构的原始顺序具有良好的局部性特征，此类排序甚至可能延缓图处理速度（Balaji& Lucia，2018）。*选择性* 枢纽排序被提出以解决此问题：仅当枢纽节点间存在密集连接时，才对图结构进行重新排序。为判断是否需对图进行排序，我们会计算“填充因子”——这一计算成本较低的诊断指标可预测枢纽排序是否可能有效（Balaji& Lucia，2018）。

Hub Clustering .枢纽聚类与枢纽排序相同，但无需在分离出高度和低度节点后对枢纽进行排序（Balaji & Lucia，2018）。枢纽聚类的主要优势在于它是一种极其轻量化的重排序算法，仅需对图进行一次遍历即可完成。

基于度值的分组。基于度值的分组（DBG）（Faldu 等人，2019）是将枢纽聚类方法扩展到多个组的一种扩展。节点根据与每个组相关的度值范围被划分为w个组。这些组按度值降序排列，但每个组内节点的顺序保持不变。（Faldu等人，2019）的作者采用对数间隔阈值，以便在幂律图中将节点均匀分配到各组中。由于 k-NN 图不具备幂律度分布特性，我们的实现则采用度分布的w个分位数。

图划分。图划分是一个已被广泛研究的领域，拥有众多成熟的算法。人们可以通过将任意图划分方法用于将节点分配到不同组中，并采用与 DBG 相同的标记方案，从而构建出一种图重排序算法。

4.2. 贪婪搜索与图重排序

在本节中，我们证明：在理想化缓存计算模型下，基于目标的图排序方法可降低贪婪搜索的复杂度。

5. 实验

实验设置。本实验采用不同重排序策略下的实际查询时间来评估索引性能。鉴于缓存未命中对多种外部因素较为敏感，我们在实验设计中特别注重构建客观的对比基准。例如，多核系统通常共享 CPU 的 L3 缓存，这可能导致缓存性能受到运行于其他核心上的无关程序的影响。

为确保获得准确且可重复的结果，我们在设计实验时力求模拟真实生产环境，并尽可能控制尽可能多的外部因素。我们在“热启动”后对系统进行基准测试，此时资产已就绪。

*表 1.*SIFT100M 上的缓存未命中与 TLB 未命中性能统计结果。我们报告的 L1、L2 及 L3 级缓存未命中数，均按硬件计数器记录的未命中次数除以数据加载总次数计算得出。

算法	L1（%）	L2（%）	L3（%）	TLB（%）
Original	19.53	13.9	6.5	3.85
RCM	17.37	**7.61**	5.1	2.56
Gorder	**14.46**	9.6	**4.0**	**2.14**

例如索引、查询及数据均预先加载至内存中。为避免因极短事件的时间控制而引发的困难，我们记录执行10,000次查询所需的总时间，并报告平均查询耗时。最后，我们将查询程序限制于单核运行，并确保基准测试程序是服务器上唯一运行的程序。所有实验均在配备252 GB内存、28个Intel Xeon E5-2697处理器及共享36 MB L3缓存的服务器上执行。

我们使用 Linux 的 perf 工具记录硬件 CPU 计数器，以测量缓存未命中率。该工具可测量数据读取和缓存命中等事件的硬件计数器。我们利用这些计数器，通过将数据缓存未命中次数除以数据访问次数，计算出缓存未命中率。我们在 SIFT100M 数据集上运行了我们最成功的重排序算法（Gorder 和 RCM），并报告了 L1、L2、L3 及 TLB 缓存的结果。为了验证这些测量结果，我们还使用 cachegrind 工具对所有重排序算法进行了实验。Cachegrind 通过虚拟处理器运行程序，记录程序执行过程中每条指令及每次缓存访问。我们对 HNSW 程序的源代码进行了注释，并报告了执行节点遍历和距离计算代码行的缓存未命中情况。

实现细节。我们扩展了 nmslib 中 HNSW 的实现，以支持图重排序。我们对索引进行了少量修改，以加速图构建并便于重排序。然而，内存访问模式及计算方面均保持不变。为确保性能数据具有实际意义，我们验证了我们的实现使用了与 nmslib- HNSW 相同的内存布局、需要相同数量的距离计算，并生成相同的图。我们采用优化的扁平化节点布局，以避免内存碎片化，并提供尽可能具有竞争力的基准性能。我们将边表示为固定大小的整数数组，并将数据与节点标签及边数组一同嵌入。

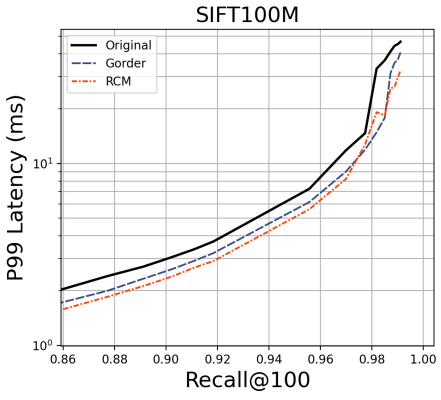


*图 3。*SIFT100M 的延迟第 99 百分位数（P99）（数值越低表示性能越好）。请注意采用了对数刻度。我们观察到使用 RCM 时性能提升 17%，使用 Gorder 时性能提升 30%。

基于目标的重排序。由于我们的k-NN搜索基准测试涉及拥有数百万节点的数据集，因此我们避开了那些已知在大规模图上扩展性能较差的算法。我们实现了Gorder和RCM，但未实现 MLOGA 或 MLINA，因为这些算法的运行时间非常高，且相较于Gorder或RCM并未带来显著改进（Wei 等，2016）。

度数 - Based Reordering。我们遵循 Balaji & Lucia（2018）的建议，将*入度*作为枢纽聚类及基于度数的方法的局部特征，因为束搜索是一种所谓的“推动式”应用[3]。我们采用平均度数作为枢纽聚类与排序的阈值，并为 DBG 使用由度数分布的 8 个分位数划分的 8 个分组。

乍一看，基于度数的重排序算法似乎不适用于近邻搜索，因为理想的k-NN图应具有恒定的度数分布。然而，这一观点并不成立。常见的图算法会将*最大*连接数设为超参数，但剪枝与多样化启发式方法生成的图在节点度数上仍会呈现显著差异。图2展示了若干真实场景中的近邻图的出度分布。尽管这些图并不严格遵循幂律分布，但其度数分布仍具备足够的多样性，因此我们有充分理由预期轻量级重排序算法可取得良好性能。

数据集。我们采用了能够代表嵌入搜索任务特性的大规模数据集。我们在 ANN-benchmarks 提供的数据集，以及 SIFT1B 和 DEEP1B 基准任务的 1000 万和 1 亿规模子集上进行了实验。表3包含了有关这些数据集的信息。在所有实验中，我们均提取查询项的前 100 个邻近样本，并报告这 100 个真实邻近样本的召回率。

超参数。在构建过程中，HNSW 需要确定两个参数：每个节点的最大边数（k_c）以及用于在图构建过程中查找k_c个邻居的 Beam 搜索缓冲区大小（M_c）。我们将M_c设置为 100，并构建索引用于

[3]关于推送/拉取应用的定义，请参阅Balaji & Lucia（2018）的第 2 节。

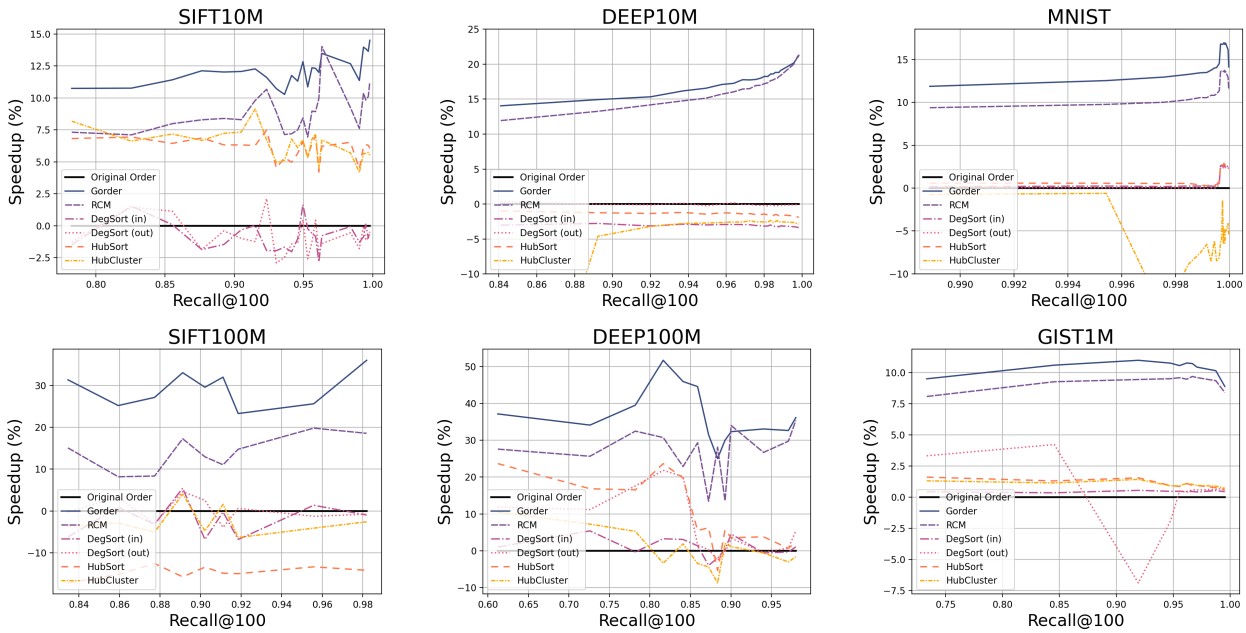


图 4。图重新排序对查询时间与召回率权衡关系的影响。位于黑色曲线之上的重新排序算法，其查询时间比原始排序更快；而位于该曲线之下的算法则会导致查询时间延长。加速效果会随召回率的变化而改变，这是因为束搜索的内存访问模式会随缓冲区大小而变化。

	最初的	戈德	RCM	降序排序（含）	降序排序（输出）	枢纽排序	枢纽集群	DBG
L1 (%)	22.76	19.28	20.61	22.76	22.76	22.85	22.77	22.81
L3 (%)	13.56	8.32	8.91	13.55	13.56	13.63	13.51	13.34

表 2。SIFT100M 数据集上节点遍历与距离计算时的缓存未命中率（数值越低越好）。需要注意的是，由于缓存与内存之间的延迟相差 100 倍，缓存未命中率的微小改善即可显著提升算法的运行速度。各算法按缓存未命中率排序的结果与图4中按加速比排序的结果一致。

*表 3.*数据集。每个数据集包含N个条目、d个特征，且每个条目需要占用“向量大小”字节。

数据集	N	d	向量大小
胃肠道间质瘤	1 M	960	3.8 kB
筛分	10 - 100 M	128	128 B
深的	10 - 100 M	96	384 B
MNIST	60 k	784	3.1 kB

$k_c \in \{4,8,16,32,64,96\}$。从这些选项中，我们选择具有最佳召回率-延迟平衡性（召回率 > 0.95）的索引，并将该索引用于图重排序实验。尽管最佳超参数确实会随召回率变化而改变，但在高召回率场景下通常存在一个明确的最佳选择。为了查询该索引， HNSW 需要指定束搜索缓冲区大小（M_q），该参数用于控制召回率与查询延迟之间的权衡。我们对图进行重排序，并对每种排序方式执行相同的 10,000 次查询。在查询该索引时，我们将束搜索缓冲区大小M_q从 100 逐步调整至 5000。

实验结果。图4展示了图重排序对查询时间与召回率权衡关系的影响。我们采用 R100@100 召回率指标，即从返回的前100个搜索结果中，统计前100个真实邻居的召回率。对于给定召回率值，我们将未重排序与重排序后的平均延迟之比定义为加速比，其中平均值基于10,000次查询及5次独立运行计算得出。图1展示了采用不同方法重排序图的耗时，并呈现重排序时间随节点数N及每个节点最大度数k_c的变化趋势。表1包含图搜索的性能缓存测量数据，表2则包含SIFT100M数据集上的缓存分析结果。由于这两项实验结果高度一致，我们可以合理推断：Gorder与RCM均通过降低缓存未命中率来缩短延迟。最后，我们在图3中测量了SIFT100M数据集的延迟第99百分位数（P99），以确保所采用的图布局改进不会加剧尾部延迟。我们发现，在高召回率条件下，Gorder 与 RCM 均能将 P99 及平均延迟时间至少提升 20%。

6. 讨论

我们的实验表明，图重排序可成为提升查询速度的标准预处理步骤，因为它不会显著增加索引构建时间。重排序仅影响节点在内存中的表示方式，而不会改变图索引的召回率、搜索算法或其他特性。基于目标的重排序算法对此问题最为有效：在小型数据集（N< 1 M）上典型加速比为 10%，而在大型数据集及高召回率场景下加速比可达 40%（图4）。Gorder 与 RCM 算法在所有实验中均持续呈现加速效果，这很可能源于重排序目标能有效模拟缓存一致性机制。可以合理推断，基于目标的方法极少（即便存在）会导致索引速度下降。然而，轻量级重排序算法在近邻搜索中的效果，远不及文献中既往研究的应用场景。尽管基于度数的聚类方法在某些任务中表现优异（如 SIFT10 M），但对于此类方法而言，近邻图往往呈现病态的度数分布。

重新排序的成本。大多数关于图重新排序的研究都聚焦于 PageRank 等应用场景——在这些场景中，图仅需被处理少量次即可获得输出结果。此类场景下的目标是实现*端到端*加速，因此会抑制过长的图重新排序耗时。这类应用更倾向于采用轻量级（但效果较差）的算法，而非更复杂的重新排序技术。然而，近邻搜索的实际运行需求恰恰相反：单个索引在其生命周期内可能被查询数百万次。近邻图构建本身已是昂贵的离线任务，因此图重新排序带来的运行时开销，对于近邻搜索而言，相较于其他应用场景而言，其影响相对较小。

尽管如此，我们观察到，在许多情况下，图重新排序时间相较于图构建时间而言可以忽略不计。例如，我们最耗时的重新排序算法（Gorder）在大多数数据集上的运行速度比 HNSW 构建速度快 10 倍（图1）。尽管 Gorder 的重新排序时间与节点度数呈二次方关系，但对于像 DEEP10M 这样具有大量相邻节点（最多可达k= 120）的大规模图索引，重新排序仍然可行。这很可能是得益于近邻图构建启发式方法，该方法通过激进剪枝降低了平均节点度数。

重新排序的优势。重新排序是一种与索引无关的方法，可用于提升基于图的近邻搜索性能。由于重新排序依赖于束搜索中的节点访问模式——而该模式在大多数算法中均普遍存在——因此重新排序可广泛应用于各类实际搜索任务。需注意的是，大多数实际部署场景均受严格延迟要求约束：将最大搜索时间控制在20毫秒内是一项常见约束条件（Nigam等，2019）。搜索时间提升20%可使系统对搜索结果进行更精细的处理，或以更高的召回率运行。

扩展。重新排序很可能与生产系统中使用的其他算法及性能优化技巧产生协同效应。图重新排序的相关思路或可应用于基于分区的搜索，因为近期一些针对局部敏感性哈希（LSH）的算法采用了k-NN 图割来构建数据分区（Dong 等人，2020）。例如，可以通过重新排列内存中的分区位置，来加速那些需要访问多个邻近分区的多探针方法。

近邻图也常与样本压缩方法结合使用，例如乘积量化或其他码本（Jegou 等人，2010）。近期关于预处理变换的实验表明，在降维子空间上执行图搜索是有益的（Prokhorenkova & Shekhovtsov，2020）。这可以增强图重排序的效果。当数据量较小时，较大的子图可容纳于 CPU 缓存中，从而提升图处理中内存局部性的优势。这一观点得到了我们实验的支持：对于 SIFT 和 DEEP，我们观察到的加速效果优于 MNIST 和 GIST，后两者的单样本存储成本较高。因此，当图重排序与量化搜索索引及降维技术结合使用时，可能展现出更大的效益。

7. 结论

我们将图重排序技术应用于基于图的近邻搜索这一广受欢迎且重要的任务中。研究发现，通过对图进行重新标记，可在查询阶段实现高达 40% 的速度提升。与图构建相比，重排序过程的计算开销较低，且我们的研究成果可广泛适用于大多数基于图的搜索索引。因此，我们预计重排序将成为基于图近邻搜索的标准预处理步骤。

参考文献

Arya，S. 与 Mount，D. M.《固定维度空间中的近似最近邻查询》。载于*SODA*，第 93 卷，第 271–280 页。Citeseer，1993。

Aumüller，M.，Bernhardsson，E. 及 Faithfull，A.《Ann-benchmarks》：一种用于近似最近邻算法的基准测试工具。*《信息系统》*，第 87 卷，第 101374 页，2020 年。

Auroux，L.，Burelle，M. 及 Erra，R. 为趣味与效益而重新排序超大规模图。载于*《国际Web算法研讨会》*，2015年。

Balaji，V. 与 Lucia，B.：图重排序何时是一种优化？研究轻量级图重排序在各类应用及输入图中的效果。见*《2018 IEEE International Symposium on Workload Characterization (IISWC)》*，第 203–214 页。IEEE，2018。

Beygelzimer，A.，Kakade，S.，与 Langford，J.《最近邻算法中的覆盖树》。载于*《第23届国际机器学习会议论文集》*，第97–104页，2006年。

Boytsov，L. 与 Naidan，B.《构建高效且高效的非度量空间检索库》。载于 Brisaboa，N.，Pedreira，O. 及 Zezula，P.（编）：*相似性搜索及其应用*，第 280–293 页，柏林、海德堡，2013年。Springer Berlin Heidelberg 出版。ISBN 978-3-642-41062-8。

Chan，T. M.《RAM 上的最近点问题简化》。载于*《第 13 届 ACM-SIAM 离散算法年度研讨会论文集》*，第 472–473 页，2002 年。

Chierichetti，F.，Kumar，R.，Lattanzi，S.，Mitzenmacher，M.，Panconesi，A.，及 Raghavan，P. 关于社交网络压缩的研究。载于*《第15届 ACM SIGKDD 知识发现与数据挖掘国际会议论文集》*，第 219–228 页，2009 年。

Connor，M. 与 Kumar，P. 针对点云快速构建 k-最近邻图。*IEEE Transactions on Visualization and Computer Graphics*，16(4):599–608,2010.

Cuthill，E. 与 McKee，J.：减少稀疏对称矩阵的带宽。In *Proceedings，1969年第24届全国会议*，第157–172 页，1969年。

Dong，W.，Moses，C.，与 Li，K. 针对通用相似度度量的高效 k-最近邻图构建方法。载于*《第20届万维网国际会议论文集》*，第 577–586 页，2011 年。

Dong，Y.，Indyk，P.，Razenshteyn，I. P.，及 Wagner，T. 用于最近邻搜索的学习式空间划分。*ICLR*，2020。

Faldu，P.、Diamond，J. 与 Grot，B.：对轻量级图重排序的深入探讨。载于*《2019 IEEE International Symposium on Workload Characterization (IISWC)》*，第 1–13 页。IEEE，2019。

Frome，A.，Corrado，G. S.，Shlens，J.，Bengio，S.，Dean，J.，Ranzato，M.，及 Mikolov，T. "Devise：一种深度视觉语义嵌入模型"。载于*《第 26 届国际神经信息处理系统会议论文集——第 2 卷》*，第 2121–2129 页，2013 年。

George，J. A.《有限元法的计算机实现》。技术报告，斯坦福大学加州分校计算机科学系，1971年。

Harwood，B. 与 Drummond，T. Fanng：快速近似最近邻图。载于*《IEEE 计算机视觉与模式识别会议论文集》*，第 5713–5722 页，2016 年。

Huang，P.-S.，He，X.，Gao，J.，Deng，L.，Acero，A.，及 Heck，L. 基于点击率数据学习用于网页搜索的深度结构化语义模型。载于*《第 22 届 ACM 信息与知识管理国际会议论文集》*，第 2333–2338 页，2013 年。

Iwasaki，M. 用于邻近搜索的剪枝双向 k-最近邻图。载于*《国际相似性搜索及其应用会议》*，第 20–33 页。Springer，2016。

Iwasaki，M. 与 Miyazaki，D. 基于 k-最近邻图的索引优化方法用于高维数据中的邻近搜索。*arXiv 预印本 arXiv:1810.07355*，2018年。

Jafari，O.，Maurya，P.，Nagarkar，P.，Islam，K. M.，及 Crushev，C.《局部敏感哈希算法及其应用综述》。*arXiv 预印本 arXiv:2102.08942*，2021年。

Jegou，H.，Douze，M.，与 Schmid，C. 用于最近邻搜索的产物量化。*IEEE 模式分析与机器智能汇刊*，33 (1):117–128,2010.

Johnson，J.，Douze，M.，and J ′egou，H. 基于 GPU 的百亿级相似性搜索。*arXiv 预印本 arXiv:1702.08734*，2017。

Johnson，J.，Douze，M.，与 J ′egou，H. 基于 GPU 的百亿级相似性搜索。*IEEE Transactions on Big Data*，2019。

Kusner，M. J.，Sun，Y.，Kolkin，N. I.，及 Weinberger，K. Q.《从词嵌入到文档距离》。载于*《第 32 届国际机器学习大会论文集——第 37 卷》*，第 957–966 页，2015 年。

Laarhoven，T. 基于图的近似近邻时空权衡研究。*arXiv 预印本 arXiv:1712.03158*，2017。

Lin，P.-C. 与 Zhao，W.-L. 关于分层可导航小世界图的比较研究。*arXiv 预印本 arXiv:1904.02077*，2019a。

Lin，P.-C. 与 Zhao，W.-L. 基于图的最近邻搜索：前景与局限。*arXiv 预印本 arXiv:1904.02077*，2019b。

Lowe，D. G. 基于局部尺度不变特征的物体识别。载于*《第七届 IEEE 计算机视觉国际会议论文集》*，第 2 卷，第 1150–1157 页。IEEE，1999。

Malkov，Y.，Ponomarenko，A.，Logvinov，A.，及 Krylov，V. 基于可导航小世界图的近似最近邻算法。*信息系统*，45: 61–68,2014.

Malkov，Y. A. 与 Yashunin，D. A. 基于分层可导航小世界图的高效且鲁棒的近似最近邻搜索。*IEEE 模式分析与机器智能汇刊*，2018.

Mikolov，T.，Sutskever，I.，Chen，K.，Corrado，G. S.，及 Dean，J. 单词与短语的分布式表示及其组合性。*《神经信息处理系统进展》*，第 26 卷：第 3111–3119 页，2013 年。

Naidan，B.，Boytsov，L.，Malkov，Y.，and Novak，D. 《非度量空间库手册》。*arXiv 预印本 arXiv: 1508.05470*，2015年。

Nigam，P.，Song，Y.，Mohan，V.，Lakshman，V.，Ding，W.，Shingavi，A.，Teo，C. H.，Gu，H.，及 Yin，B. 语义产品搜索。载于*《第25届ACM SIGKDD 知识发现与数据挖掘国际会议论文集》*，第 2876–2885 页，2019 年。

Ondov，B. D.，Treangen，T. J.，Melsted，P.，Mallonee，A. B.，Bergman，N. H.，Koren，S.，及 Phillippy，A. M. 《Mash：基于 minhash 的快速基因组与宏基因组距离估算》。*《基因组生物学》*，17(1):1–14,2016。

普罗霍连科娃，L. 与谢霍夫佐夫，A. 基于图的最近邻搜索：从实践到理论。载于*《国际机器学习会议》*，第 7803–7813 页。 PMLR ，2020。

Safro，I. 与 Temkin，B. 面向网络压缩友好排序的多尺度方法。*《离散算法杂志》*，第 9 卷第 2 期：第 190–202 页，2011 年。

Sebastian，T. B. 与 Kimia，B. B. 《大型数据库中的基于度量标准的形状检索》。载于《*面向服务机器人的用户交互支持下的目标识别*》，第3卷，第291–296页。IEEE，2002年。

Shakhnarovich，G.，Darrell，T.，及 Indyk，P. 学习与视觉中的最近邻方法。*IEEE Trans. Neural Networks*，19 (2):377,2008.

Shimomura，L. C.，Oyamada，R. S.，Vieira，M. R.，及 Kaster，D. S. 《度量空间中基于图的相似度检索方法综述》。*《信息系统》*，第 101507 页，2020 年。

Wei，H.、Yu，J. X.、Lu，C. 与 Lin，X.：通过图排序加速图处理。《2016 年 International Conference 数据管理论文集》第 In Proceedings 卷，第 1813–1828 页，2016 年。

Zhang，Y.、Kiriansky，V.、Mendis，C.、Amarasinghe，S. 与 Zaharia，M.：实现缓存在图分析中的应用。载于*《2017 IEEE International Conference on Big Data (Big Data)》*，第 293–302 页。IEEE，2017。